

01 - Ambienti distribuiti

Si definisce in maniera rigorosa e non ambigua il *modello di calcolo* sul quale verranno studiati gli **algoritmi distribuiti**, anche detti **protocolli**. Tale modello prende il nome di **ambiente distribuito** ed è utile non solo per definire algoritmi distribuiti, ma anche per l'analisi della loro *correttezza* e *stabilità*.

Informalmente, **un ambiente distribuito è composto da un insieme di agenti** aventi ognuno delle **capacità computazionali**, che interagiscono tra di loro scambiandosi dei messaggi (*message passing*) tramite una serie di **interconnessioni**, andando a formare quindi una rete. Tale ambiente distribuito **viene modellato mediante grafo**, in particolar modo, data una rete costituita da agenti e interconnessioni tra di essi, il grafo sottostante ha un vertice per ciascun agente del sistema e un arco tra due nodi per ogni interconnessione tra i corrispettivi agenti.

Entità

Un ambiente distribuito è composto da entità (o agenti, nodi). Ogni nodo $x \in \mathcal{E}$ della rete, dove $\mathcal{E}$ è l'insieme degli agenti, ha delle capacità computazionali, le quali sono definite come un insieme di **operazioni possibili**, tra le quali:

- **Local storage e processing**, ovvero la capacità di conservare informazioni e di processarle **localmente**;
- **Scambio di messaggi** con altri nodi della rete;
- **(re-)impostare il clock**, infatti ogni nodo ha un **clock** interno che scandisce le esecuzioni delle sue operazioni, che può essere impostato dal nodo stesso;
- **Cambiare il valore dei registri**, infatti ogni nodo ha dei **registri** usati per eseguire le operazioni e/o mantenere uno **stato** (generalmente viene utilizzato un **registro di stato** apposito).

operazioni
possibili di un nodo

Stato di un entità

Ogni entità ha uno **stato** interno, il quale rappresenta un'informazione *necessaria* per il corretto funzionamento di un protocollo distribuito.

Formalmente, **ogni nodo ha un insieme finito di stati possibili Σ sempre definito a priori, e in ogni istante un'entità deve trovarsi in uno dei suoi possibili stati.**

Ad esempio, un insieme di stati può essere il seguente

$$\Sigma = \{\text{idle, computing, waiting, ...}\}$$

Eventi

In un ambiente distribuito, **il comportamento di ogni entità è reattivo**, ovvero è indotto a seguito del verificarsi di **eventi**. **In assenza di stimoli, un'entità rimane inerte.**

In generale questi eventi possono essere:

- interni al nodo;
 - esterni al nodo ma interni all'ambiente distribuito, provenienti quindi da altre entità;
 - esterni dall'ambiente distribuito.
- Alcuni esempi di possibili eventi che modificano il comportamento di un'entità sono:
- il *tick* del clock (evento interno al nodo);
 - la ricezione di un messaggio spedito da un altro agente (evento interno alla rete);
 - un'impulso spontaneo (evento esterno alla rete).

eventi ai quali un
nodo può reagire

Azioni

Un'entità *reagisce* in seguito ad un evento. In particolare, l'azione che un nodo compie in risposta ad un evento dipende anche dal suo stato interno nel momento dello stimolo.

Dunque una combinazione di evento e stato interno implicano un'azione da parte di un nodo

$$\text{stato} \times \text{evento} \mapsto \text{azione}$$

dove con **azione** si intende una *sequenza di task* (o attività) da svolgere, come ad esempio

- computare un algoritmo;
- inviare dei messaggi;
- cambiare lo stato interno.

In questo modello, ogni azione è **atomica e finita**:

- **atomica**: una volta avviata, un'azione non può essere interrotta;
- **finita**: un'azione termina sempre in tempo finito.

Si osserva che un'entità in un certo stato potrebbe non reagire ad un'evento: in questo caso si assume che l'entità ha eseguito l'azione *nil* o azione nulla.

AZIONI DI UN
NODO

Comportamento delle entità

Le associazioni delle coppie stato-evento con la relativa azione sono dette **regole**.

Data un'entità x , sia Σ il suo insieme di stati possibili, sia $\hat{E}$ l'insieme degli eventi e sia A l'insieme delle azioni che questa può eseguire.

In una regola $s \times e \rightarrow a$, dove $s \in \Sigma$, $e \in \hat{E}$ ed $a \in A$, si dice che a è attivata dalla coppia (s, e) .

In generale, si definisce il **comportamento** di un'entità come un insieme di regole, definite per ogni possibile coppia stato-evento. Il **comportamento** di x è descrivibile quindi mediante l'insieme $B(x)$, contenente tutte le regole che x rispetta, che consiste proprio nel protocollo di x .

Tale insieme deve essere **non-ambiguo e completo**, ossia, ad ogni possibile coppia stato-evento deve essere associata un'unica azione:

$$\forall (s, e) \in \Sigma \times \hat{E}, \exists! a \in A \text{ tale che } s \times e \rightarrow a$$

Comportamento del protocollo

Raggruppando tutti i comportamenti delle entità di un sistema è possibile definire il **protocollo (o algoritmo) distribuito**.

Formalmente, un protocollo è l'insieme

$$B(\mathcal{E}) := \{B(x) : x \in \mathcal{E}\}$$

Un sistema si dice **simmetrico (o omogeneo o anonimo)** se tutte le entità hanno lo stesso comportamento

$$B(x) \equiv B(y) \quad \forall x, y \in \mathcal{E} \quad \text{tutti gli agenti si comportano allo stesso modo}$$

In questo caso, per specificare il comportamento collettivo del sistema, risulta sufficiente specificare il comportamento di una singola entità.

Si osserva che è possibile trasformare ogni sistema asimmetrico in un sistema simmetrico.

Questo significa che se le entità in un sistema eseguono azioni differenti, a seconda del proprio ruolo, in risposta alla stessa coppia stato-evento, è sempre possibile scrivere un nuovo insieme di regole, uguale per tutti, che li farà comportare come nella variante asimmetrica.

Si consideri il seguente esempio: nel sistema esistono due tipologie di entità: *server* e *client*, che hanno insieme di regole differenti. In particolare, per ogni coppia $(s, e) \in \Sigma \times \hat{E}$, i nodi client hanno la regola

$$(s, e) \mapsto a_{\text{client}} \in A_{\text{client}}$$

dove A_{client} è l'insieme di azioni dei nodi client, mentre i nodi server hanno la regola

$$(s, e) \mapsto a_{\text{server}} \in A_{\text{server}}$$

dove A_{server} è l'insieme di azioni dei nodi server.

Allora è possibile fondere i comportamenti dei due tipi di entità creando un unico insieme di regole in cui l'azione eseguita dipende dalla tipologia dell'entità. In particolare, le nuove regole sono del tipo

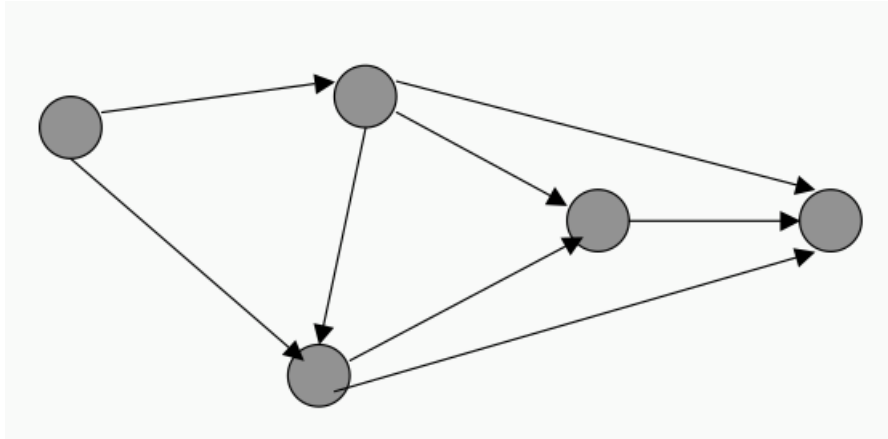
$$(s, e) \mapsto \{\text{if role} = \text{client then } a_{\text{client}} \text{ else } a_{\text{server}}\}$$

Se la coppia (s, e) non costituisce nessuna regola in per il client (analogo server) allora nel nuovo insieme di regole l'azione associata a tale coppia avrà $a_{\text{client}} = \text{nil}$ (analogamente, $a_{\text{server}} = \text{nil}$).

Comunicazione

Le entità della rete distribuita possono e devono interagire tra di loro. Ciò avviene tramite **scambio di messaggi (o message passing)**, dove un messaggio è inteso essere una **sequenza finita di bit**.

I messaggi viaggiano su una **rete di comunicazione** che può essere modellata per mezzo di un **grafo diretto** $G = (V, E)$, dove $\mathcal{E} = V$, che specifica, per ogni nodo, le entità con cui questo può comunicare.

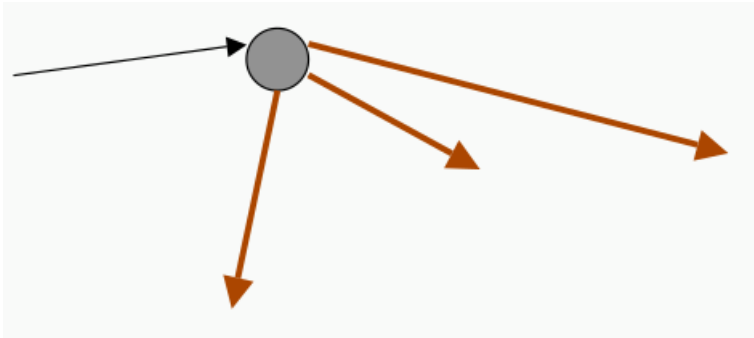


Si osserva che i nodi della rete non hanno una visione *globale* della rete, cioè non conoscono la topologia del grafo, bensì hanno solo una visione *locale*. Formalmente, ogni nodo x vede solamente due gruppi di porte logiche: le porte dalle quali arrivano i messaggi in entrata e le porte dalle quali vengono mandati i messaggi in uscita:

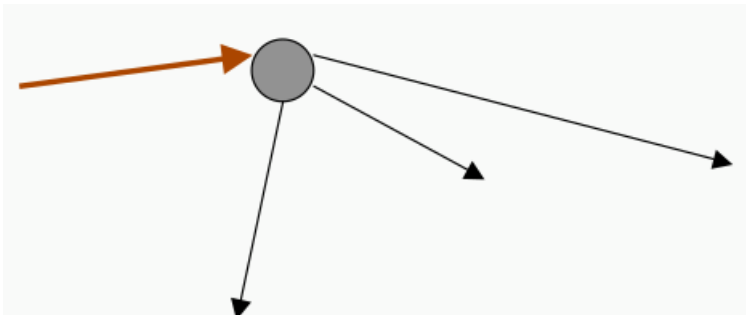
- $N_{\text{out}}(x) \subset V$: insieme dei vicini *uscenti* dell'entità $x \in V$;
 - $N_{\text{in}}(x) \subset V$: insieme dei vicini *entranti* dell'entità $x \in V$;
- In particolare, il *vicinato* di un nodo x è l'insieme

$$N(x) = N_{\text{out}}(x) \cup N_{\text{in}}(x)$$

Dunque, un nodo x può inviare i messaggi solamente ai suoi vicini uscenti in $N_{\text{out}}(x)$,



e ricevere messaggi dai suoi vicini entranti in $N_{\text{in}}(x)$



I messaggi ricevuti da un nodo vengono processati nell'ordine di arrivo, che non necessariamente coincide con l'ordine in cui questi sono stati spediti dagli altri agenti in rete. Se più di un messaggio viene ricevuto nello stesso istante, questi verranno processati in ordine arbitrario. Le entità ed i canali di comunicazione possono subire guasti.

Assiomi della comunicazione

Ogni argomento verrà trattato sotto i seguenti assiomi riguardo la comunicazione.

Finite transmission delays

In assenza di perdite di messaggi, si assume che il tempo di trasmissione di un messaggio da un nodo verso un suo vicino raggiungibile da un arco uscente sia **finito**. Formalmente, per ogni messaggio m esiste un tempo $t \in \mathbb{R}^+$ tale per cui, entro il tempo t , il messaggio m viene trasmesso lungo il canale di comunicazione.

Si osserva che tale assioma non implica l'esistenza di un qualsiasi bound sul tempo di trasmissione: afferma solo che in assenza di perdite di messaggi, un messaggio raggiunge la propria destinazione in una quantità di tempo finita senza subire corruzioni, e quindi, tale tempo di trasmissione può essere non noto.

Local orientation

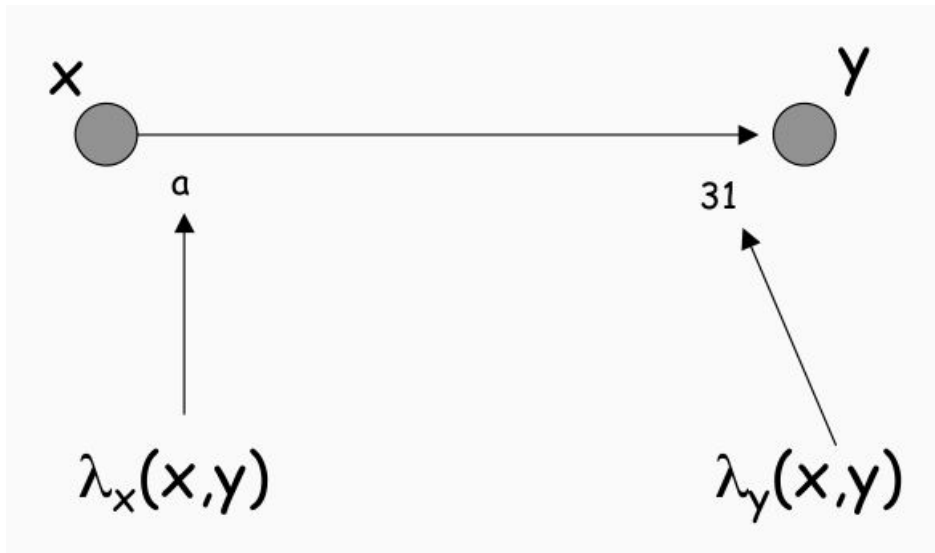
Un'entità può comunicare direttamente con un sottoinsieme delle entità: l'insieme dei suoi vicini. Ogni nodo *distingue* i proprio vicini, ossia:

- Un'entità è in grado di distinguere i suoi vicini entranti.
 - Un'entità è in grado di distinguere i suoi vicini uscenti.
- In particolar modo:
- Un'entità è sempre in grado di inviare un messaggio solamente ad uno specifico e singolo vicino uscente, cioè non deve necessariamente inviare il messaggio attraverso tutti i suoi archi uscenti;
 - Un'entità è sempre in grado di distinguere quale dei suoi vicini gli ha inviato un messaggio;

In altre parole, ogni entità x ha una funzione locale iniettiva $\lambda_x : E_x \rightarrow L$, dove $E_x := \{(u, v) \in E \mid u = x \vee v = x\}$ è l'insieme degli archi incidenti ad x , ed L un insieme di etichette (ad esempio stringhe, interi), che permette quindi tale agente di assegnare un'etichetta a ciascun arco ad esso incidente.

Si osserva esplicitamente che, per ogni arco orientato (x, y) , esistono due etichette: $\lambda_x(x, y)$ locale al nodo x , e $\lambda_y(x, y)$, locale al nodo y .

ogni nodo distingue ogni sua porta con una etichetta distinta e not



Si evidenzia che, dato un arco (x, y) , l'etichettatura $\lambda_x(x, y)$ è locale ad x , e non necessariamente coincide con l'etichetta che assegna y a tale arco. Ossia, in generale, vale:

$$\lambda_x(x, y) \neq \lambda_y(x, y)$$

Inoltre, in generale, si ha

$$\lambda_x(x, y) \neq \lambda_x(y, x)$$

ovvero, un x può non sapere che (x, y) e (y, x) sono archi che lo collegano alla stessa entità y .

Per questa definizione di orientamento locale, si farà sempre riferimento a grafi *etichettati* (G, λ) , dove $\lambda = \{\lambda_x : x \in V\}$.

Restrizioni del modello

Spesso i protocolli vengono definiti e studiati assumendo ulteriori restrizioni. Naturalmente un protocollo definito sotto un gran numero di restrizioni è generalmente *debole*, nel senso che, rimuovendo alcune di queste restrizioni, il protocollo potrebbe non risolvere il problema preso in analisi. Un protocollo definito su un sistema con poche restrizioni invece risulta più *generale*, e dunque più forte.

Si elencano alcune tipologie di restrizioni.

Message ordering

Spesso si assumerà che, in assenza di perdita di messaggi, i messaggi trasmessi mediante uno stesso link arrivano nello stesso ordine. Seguono quindi una coda di tipo **FIFO**, ordinata con l'ordine di invio dei messaggi.

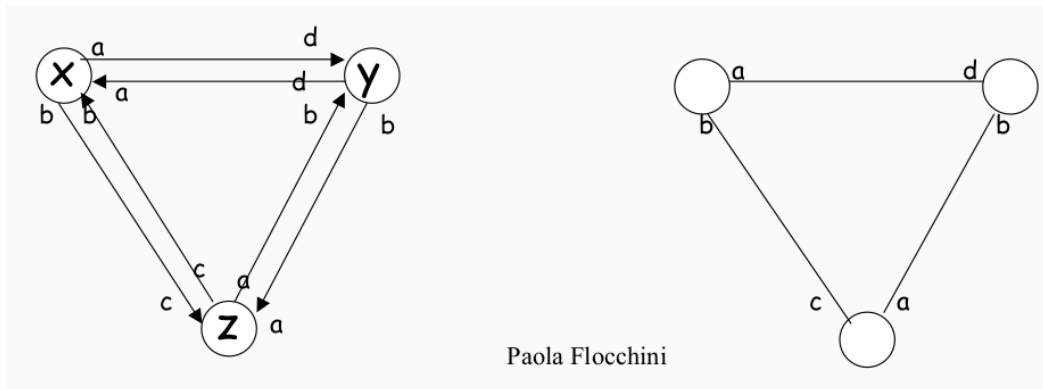
Si osserva esplicitamente che la Message Ordering non implica né l'esistenza di un

ordinamento tra messaggi trasmessi alla stessa entità da archi distinti, né per messaggi mandati dalla stessa entità su archi distinti.

Link bidirezionali

Questa restrizione assume che le connessioni nella rete sono *bidirezionali*, ossia, $\forall x, y \in \mathcal{E}$, se $(x, y) \in E$ allora $(y, x) \in E$, e $\forall x \in \mathcal{E}$ l'etichetta che x assegna agli archi $(x, y), (y, x)$ è uguale, formalmente

$$\forall x \in V, N_{\text{out}}(x) = N_{\text{in}}(x) = N(x) \wedge \forall y \in N(x), \lambda_x(x, y) = \lambda_x(y, x)$$



dunque **ogni nodo è in grado di distinguere gli estremi degli archi**. Tale restrizione implica la possibilità di modellare la rete come un grafo non diretto.

Restrizioni sulla disponibilità

1. **Delivery garantita.** Ogni messaggio che viene inviato viene ricevuto correttamente, cioè non ci sono corruzioni di messaggi. Sotto questa restrizione, i protocolli non devono tenere conto di omissioni o corruzioni dei messaggi durante la trasmissione.
2. **Disponibilità parziale.** Non si verificano guasti (come ad esempio, perdite di messaggi) dall'istante in cui si inizia ad osservare il sistema. Si osserva che, negli istanti precedenti all'inizio dell'osservazione, il sistema potrebbe aver subito dei guasti.
3. **Disponibilità totale.** Non sono mai avvenuti fallimenti in passato e mai avverranno in futuro.

Restrizioni sulla topologia

Sono tutte quelle assunzioni che si possono fare sulla struttura del grafo G che descrive la rete di comunicazione del sistema. Ad esempio, spesso si assume che G è fortemente connesso. Quest'ultima proprietà prende il nome di *connectivity*.

Restrizioni sulla conoscenza

Sono tutte quelle assunzioni che riguardano la conoscenza da parte dei nodi di informazioni *globali*, cioè di informazioni che riguardano il sistema distribuito. Ad esempio, si può

cosa sanno i nodi su delle variabili globali?

assumere che sia disponibile come informazione globale il numero dei nodi della rete, il numero dei links, il valore del diametro della rete, ecc.

Restrizioni temporali

Eccetto il tempo di trasmissione finito, il modello generale non fa assunzioni su tale quantità.

Due possibili restrizioni riguardo i tempi di comunicazione sono le seguenti:

- **Bounded communication delay.** Esiste una costante Δ tale che, in assenza di guasti, il tempo di trasmissione dei messaggi tramite qualsiasi link è al più Δ ;
- **Clock sincronizzati.** Ogni clock locale di ciascuna entità è incrementato di una unità temporale in modo *simultaneo*, e l'intervallo di tempo tra gli incrementi successivi è costante.

il clock globale è uguale $\forall$ nodo $c \in G$

Misure di complessità

In generale in un sistema distribuito non ha sempre senso parlare di *complessità temporale*, in quanto non è detto che il tempo di trasmissione di un messaggio sia sempre lo stesso, e non è detto che il clock sia uguale per tutti i nodi. Per far sì che abbia senso parlare di *time complexity* è necessario imporre due restrizioni temporali: la *synchronized clocks* e la *Bounded Communication Delay*. Idealmente, tutti i messaggi dovrebbero viaggiare con uno stesso delay unitario.

Un altro tipo di complessità presa in analisi è la cosiddetta *message complexity*, ovvero una misura che tiene conto del numero di messaggi trasmessi prima che un protocollo termini un suo *task*. Si osserva che, se un messaggio per arrivare dal nodo a al nodo b necessita di venir instradato k volte, allora vengono contate tutte le k trasmissioni fatte prima di arrivare a b , anche se il messaggio è lo stesso che attraversa la rete. Inoltre, anche un messaggio non ricevuto a causa di un guasto viene considerato nel conteggio.